Cyber-Physical Systems and IoT Security - First Project

Alberto Penzo 2109317

Reference Paper: Error Handling of In-vehicle Networks Makes Them Vulnerable

1. Objectives

This project reproduces and validates the bus-off attack through a software simulation implemented in C. The attack exploits CAN's built-in mechanism to forcibly disconnect a healthy ECU from the in-vehicle network by manipulating its Transmit Error Counter (TEC) past the bus-off threshold (TEC > 255).

The specific objectives for this project are:

- Implement a faithful software model of the CAN error-handling state machine (Error-Active -> Error-Passive -> Bus-Off) in C, following the CAN 2.0 specification.
- Simulate the full two-phase iterative bus-off attack (Phase 1: both nodes in Error-Active; Phase 2: victim in Error-Passive, adversary returns to Error-Active) as described in the paper.
- Show the evolution of TEC for both the victim and adversary nodes over time, including the phase transition and the bus-off event.
- Show the difference in the attack successfulness between period-based vs preceded ID based synchronization when there is a random jitter in the simulation.

2. System Setup

No physical hardware or commercial tools were used. The software is a custom-built, discrete-event C simulation.

2.1 Software Architecture

The simulator is structured into three modules:

- ecu.h / ecu: TEC/REC counters, fault-confinement state machine and node scheduling fields
- can_bus.h / can_bus.c: bus topology, discrete-event simulation loop, attack logic
- main.c: node initialization, simulation entry point, CSV output and reporting

2.2 CAN State Machine Parameters

The following CAN specification constants are encoded:

- TEC increment on Tx error +8
- TEC decrement on Tx success -1
- Error-Passive threshold >127
- Bus-off threshold >255
- Bus speed 500Kbps
- Frame duration 200 ms
- Error-Frame window 34 μs

- Intra-Frame Space 6 μs
- Message period 10 ms

2.3 Node Configuration

Three ECU nodes are instantiated in main.c to replicate the scenario described in section 5.1 of the paper:

- Node A (ID=0x07 & ID=0x09): a background ECU that generates the preceded ID for the victim's message. It does not participate in the attack directly but serves to populate the bus with realistic traffic.
- Victim Node (ID=0x11, DLC=1): the legitimate ECU targeted by the attack. It transmits periodically every 10 ms.
- Adversary Node (ID=0x11, DLC=0): the compromised ECU. It is configured via ecu_make_adversary() to target ID 0x11 and sends all-dominant bits (DLC=0).

3. Experiments

3.1 Attack Simulation Logic

The simulation is implemented in bus_simulate_attack() inside can_bus.c. The function advances a microsecond-resolution simulation clock and calls the ECU state-machine helpers (ecu_on_tx_error(), ecu_on_tx_success()) to apply the exact TEC delta rules defined by the CAN specification. The attack unfolds in three phases.

Phase 1 - Both nodes in Error-Active

The adversary injects an attack message with the same ID as the victim but with DLC=0 (all dominant bits). Both nodes win arbitration simultaneously. The victim detects a bit error and its TEC increases by +8. Its Active Error Flag (6 dominant bits) violates the bit-stuffing rule at the adversary, so the adversary's TEC also increases by +8. The CAN controller triggers an automatic retransmission, and the collision repeats without any further action by the adversary. This loop continues until both nodes reach TEC = 128 (16 retransmissions, ~3.7 ms).

Phase 1 2 - Transition

At TEC = 128 both nodes become Error-Passive. One additional collision occurs. The victim's TEC rises to 136; it attempts a Passive Error Flag (6 recessive bits) which is overridden by the adversary's dominant frame and goes unacknowledged. The adversary completes its frame successfully (TEC: 128 → 127, returning to Error-Active). The victim retransmits without further interference (TEC: 136 → 135, remaining Error-Passive).

Phase 2 - Victim in Error-Passive, adversary in Error-Active

On every subsequent 10 ms scheduled transmission of the victim, the adversary attempts to re-inject its attack message. Without jitter, synchronization is guaranteed every cycle: victim bit error (+8), adversary success (-1), victim retransmission success (-1), for a net victim TEC change of +7 per cycle.
With jitter, the outcome depends on the adversary’s synchronization strategy. Under period-based timing, the adversary predicts the victim’s transmission using the nominal period

but cannot track the actual deviation; a collision only occurs when the jitter falls within one bit time (2 µs), roughly 6.7% of cycles at ±30 µs. Missed cycles yields no TEC increase for the victim and no TEC decrease for the adversary, stalling the attack.
Under preceded ID timing, the adversary derives its attack instant from the completion of a known background message, which is subject to the same jitter as the victim; synchronization is thus reserved across cycles and the +7 net TEC progression is maintained.
The victim inevitably crosses TEC = 255 and disconnects from the bus.

3.2 Measurement and Data Collection

A sim_record_t structure (time_us, victim_tec, adv_tec, victim_state, adv_state) is recorded at every state-change event. The simulation runs for a hard limit of 200 ms, which is sufficient to observe bus-off.

The simulation is run 3 times in 3 different scenarios.

Run 1 Period based attack - no jitter

Run 2 Preceded ID based attack with jitter

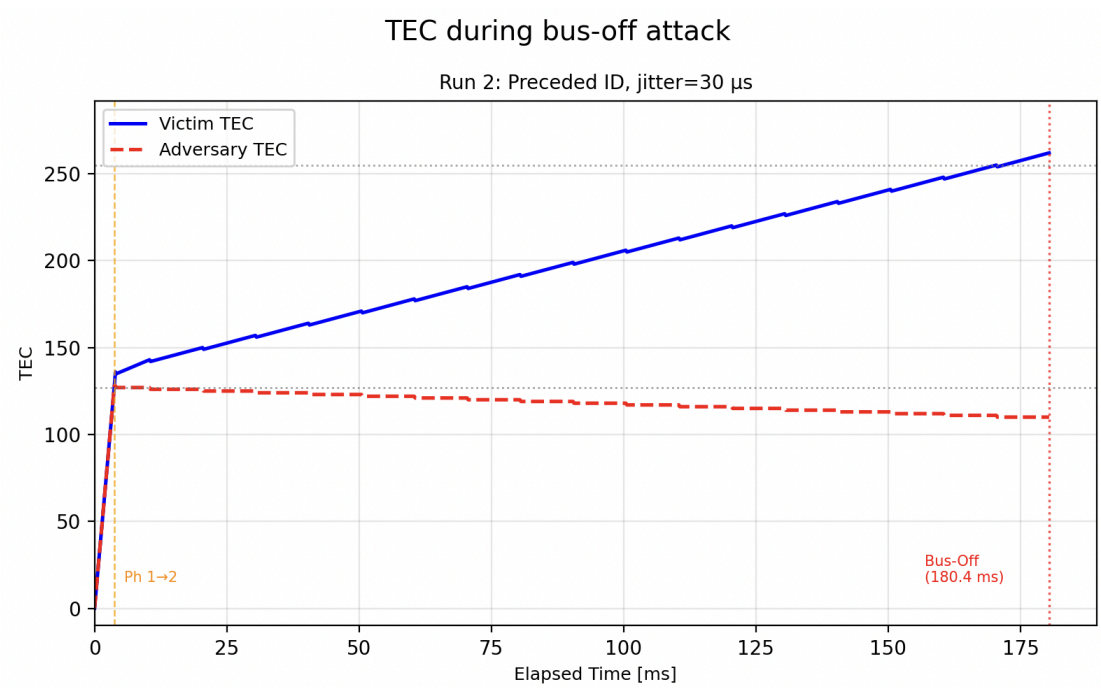
Run 3 Period-based attack with jitter

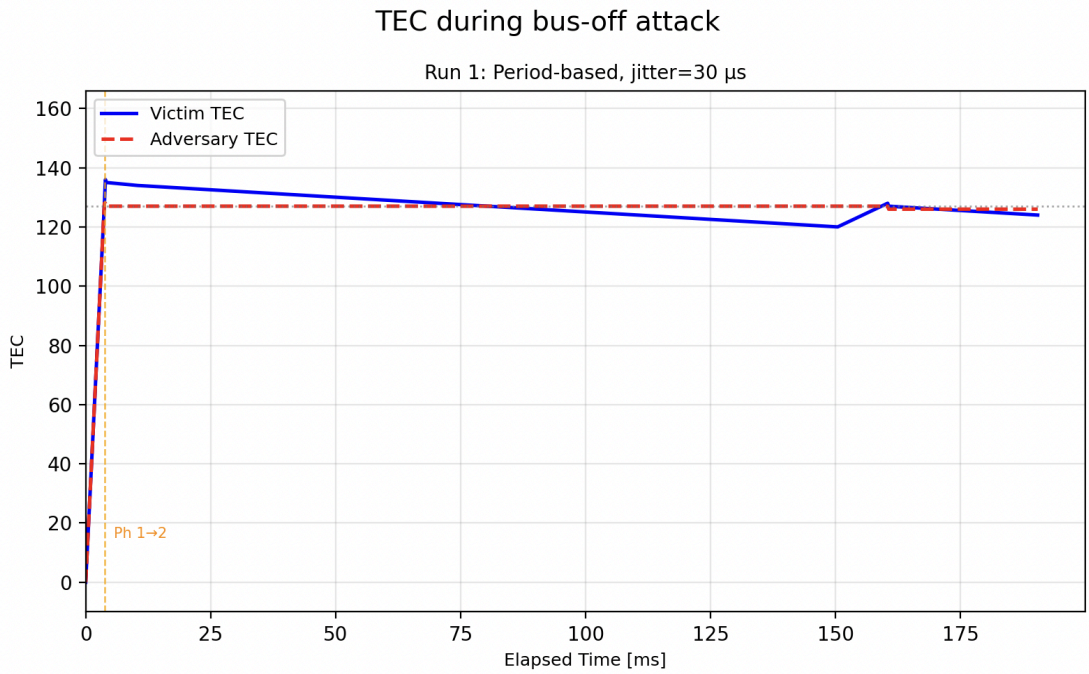
A Python script (plot.py) reads the CSVs (run 2 and 3) and produces a TEC evolution plot showing the two different attack strategies in the presence of jitter, annotating the Phase 1 to 2 boundary and the bus-off event.

4. Results and Discussion

4.1 Reproduced Figure

Figure 1 below reproduces Figure 12 from the reference paper. The TEC trajectories of both nodes are plotted across the full 180 ms simulation window.





The steep rise in Phase 1 (both TECs increasing to 128), the asymmetric divergence following the Phase 1-2 transition, and the monotonically rising victim TEC in Phase 2 with its net +7 sawtooth increments are all visible.

Figure 2 shows that the period-based attack is inefficient at disconnecting the victim ECU from the bus if the jitter is too big. The hit chance is too low to keep up with the TEC decrease from successful victim's transmissions and at the end of the simulation (200 ms) the victim has not reached bus-off.

4.2 Discussion

The simulation results confirm the claims of the reference paper:

Agreement on Phase 1 duration. The paper states that with a single attack injection, the victim enters Error-Passive in less than 10 ms. The simulation reports a Phase 1 duration of 3.74 ms (16 automatic retransmissions at 500 Kbps), consistent with the experimental result of the paper.

Minor timing discrepancy at bus-off. The paper reports a bus-off delay of approximately 188 ms (Figure 14, 500 Kbps). The simulation reaches bus-off at 180 ms. The difference arises because the simulation uses simplified, fixed frame-duration constants (FRAME_DURATION_US = 200 µs) rather than computing exact per-message bit counts with stuffing overhead. Since the difference is about 4% it is an acceptable simplification.

Confirmation of attack severity. The simulation demonstrates that the adversary requires no knowledge of message content or checksums, only the ability to transmit on the same CAN ID at the same time. Once Phase 1 is initiated with a single message injection, the CAN controller's automatic retransmission mechanism drives the victim to Error-Passive autonomously. In Phase 2, only one attack message per 10 ms period is needed.

Scope of the simulation. The current implementation models the core TEC/REC state machine, the two-phase attack sequence, jitter modeling and preceded ID sync. Elements not reproduced include, multi-node bus traffic, varying bus speeds and bus loads, and the countermeasure evaluation.